Fabian Figueroa

Professor Stephen Pena

Applied Linear Algebra

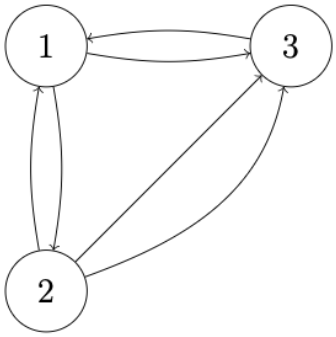
May 2, 2025

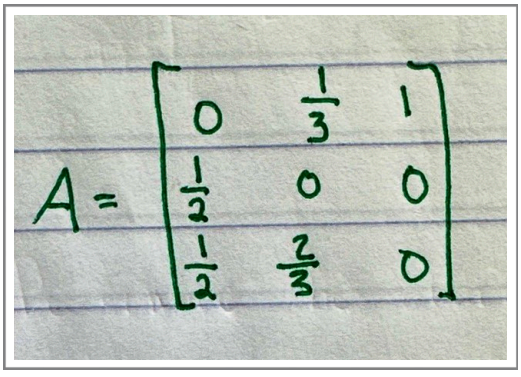
Project 2

Part 1:

(a). The transition matrix *A* corresponding to Figure 1:

Figure 1: A directed graph.

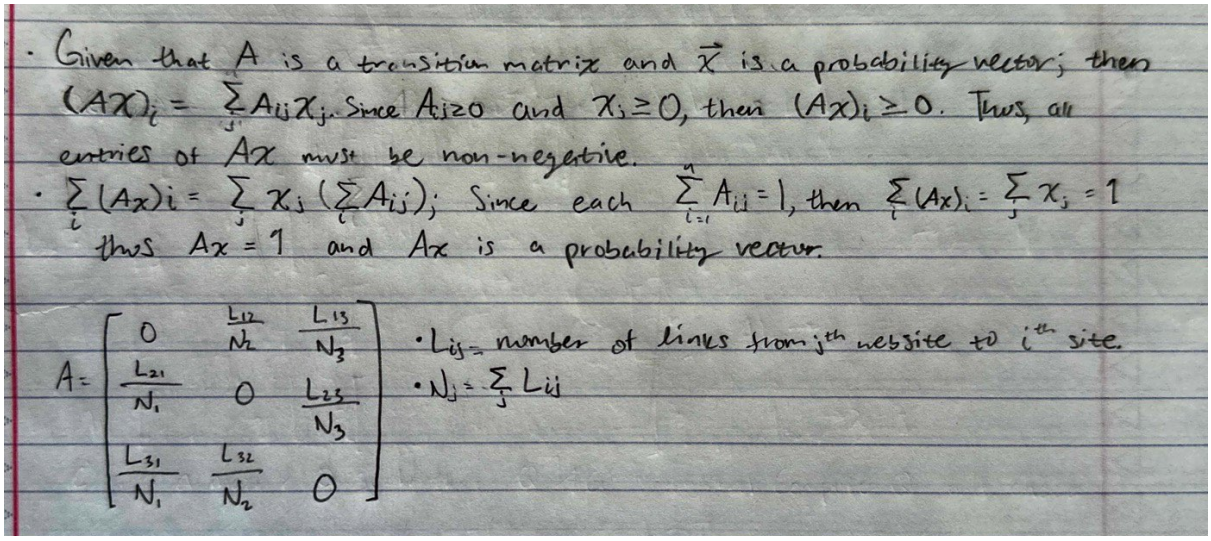




Handwritten transition matrix *A*

Each column of transition matrix *A* represents the probability distribution of outgoing links from each page. Page 1 (*col 1*) splits its probability equally between pages 2 and 3. Page 2 distributes 1/3 probability to page 1 and 2/3 to page 3; based on the number of links in the directed graph in Figure 1. Page 3 sends 100% of its probability to page 1, as it has only one outgoing link.

(b). Handwritten proof showing that for any such vector, Ax is a vector of probabilities whose sum is equivalent to 1:



Part 2:

(a). Code snippet representing the user's ability to specify the 3x3 matrix A and starting vector x_0:

```python
# --- Main Program ---
def main():
    # 1. User Input
    A = np.zeros((3, 3))
    print("Enter the elements of matrix A row by row (3x3 matrix):")
    for i in range(3):
        for j in range(3):
            while True:
                try:
                    user_input = input(f"Enter A[{i+1},{j+1}]: ")
                    A[i, j] = float(eval(user_input))
                    break
                except:
                    print("Invalid input. Enter a number or fraction like 1/2.")

    x0 = np.zeros(3)
    print("Enter the 3 elements of starting vector x0:")
    for i in range(3):
        while True:
            try:
                user_input = input(f"Enter x0[{i+1}]: ")
                x0[i] = float(eval(user_input))
                break
            except:
                print("Invalid input. Enter a number or fraction like 1/2.")
```

(b). Code snippet that determines whether the user's chosen transition matrix is valid (the columns sum to 1 and the elements are nonnegative).

```
5   def is_valid_transition_matrix(A):
6       A = np.array(A)
7       if (A < 0).any():  # allow zeros, reject negatives
8           return False
9       col_sums = np.sum(A, axis=0)
10      return np.allclose(col_sums, np.ones(A.shape[1]))
11
12  def is_probability_vector(x):
13      x = np.array(x)
14      if (x < 0).any():
15          return False
16      return np.isclose(np.sum(x), 1)
```

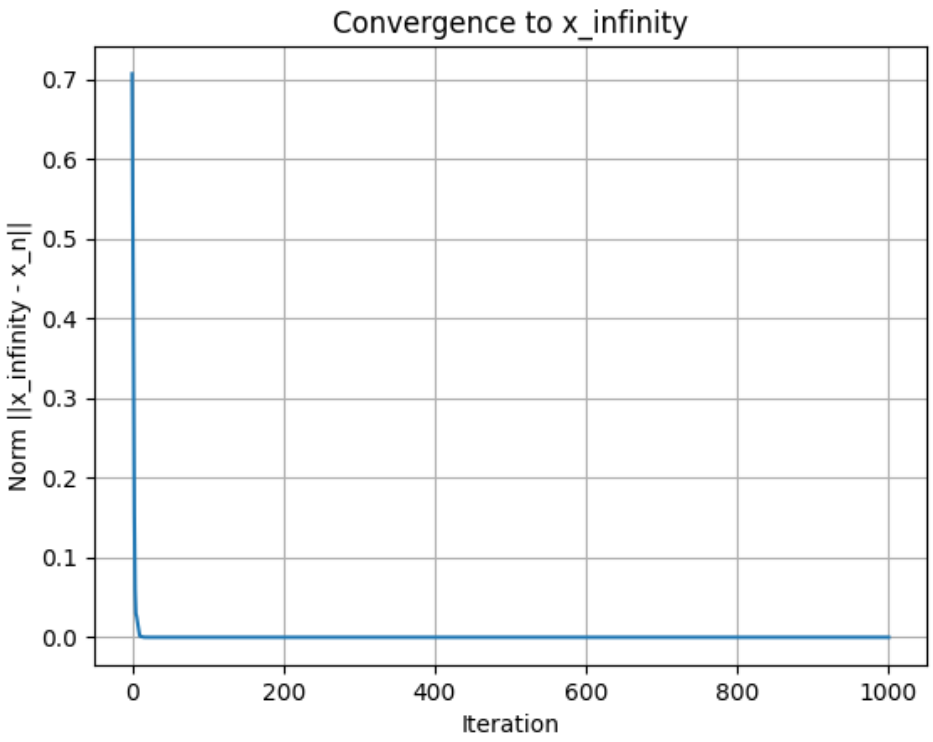
(c). Code snippet used to compute the sequence of vectors using the following general formula: $x_{n+1} = A x_n \quad (n = 0, \ldots, N)$

```
64      for _ in range(N):
65          next_vector = A @ current_vector
66          assert is_probability_vector(next_vector)
67          history.append(next_vector.copy())
68          current_vector = next_vector
```

(d). Code snippet to approximate x_∞, compute Ax_∞, and plot a graphical demonstration that $\lim_{n\to\infty} x_n = x_\infty$:

```
    print("\nApproximate x_infinity:", x_infinity)
    print("A x_infinity:", A @ x_infinity)

    # 4. Plot convergence
    norms = [np.linalg.norm(x_infinity - x) for x in history]
    plt.plot(norms)
    plt.xlabel('Iteration')
    plt.ylabel('Norm ||x_infinity - x_n||')
    plt.title('Convergence to x_infinity')
    plt.grid(True)
    plt.show()
```



(e). Code snippet to compute the eigenvalue(s) and eigenvector(s) of matrix A:

```
# 5. Eigenvalues and Eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(A)
print("\nEigenvalues:", eigenvalues)
print("Eigenvectors:\n", eigenvectors)
```

Computation utilising the NumPy library's *linalg.eig* function

```
Eigenvalues: [ 1. +0.j         -0.5+0.28867513j -0.5-0.28867513j]
Eigenvectors:
 [[-0.71713717+0.j         -0.70710678+0.j         -0.70710678-0.j        ]
 [-0.35856858+0.j          0.53033009+0.30618622j  0.53033009-0.30618622j]
 [-0.5976143 +0.j          0.1767767 -0.30618622j  0.1767767 +0.30618622j]]
```

Computation using the matrix A corresponding to the directed graph in Figure 1

(f). Various transition matrix A inputs, starting vectors, and corresponding outputs for testing purposes. Along with hypothesis about relationship between the vector x_∞ and the eigenvalues of matrix A:

$$A_1 = \begin{bmatrix} 0 & 1/3 & 1/5 \\ 0 & 1/3 & 3/5 \\ 1 & 1/3 & 1/5 \end{bmatrix} \text{ and starting vector } x_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

```
Approximate x_infinity: [0.2083 0.375  0.4167]
A x_infinity: [0.2083 0.375  0.4167]

Eigenvalues: [-0.2333+0.2809j -0.2333-0.2809j  1.    +0.j    ]
```

Output corresponding to A_1 and x_1

$$A_2 = \begin{bmatrix} 1/6 & 1/2 & 2/5 \\ 4/6 & 0 & 3/5 \\ 1/6 & 1/2 & 0 \end{bmatrix} \text{ and starting vector } x_2 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

```
Approximate x_infinity: [0.3559 0.3898 0.2542]
A x_infinity: [0.3559 0.3898 0.2542]

Eigenvalues: [ 1.     -0.216  -0.6174]
```

Output corresponding to A_2 and x_2

$$A_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ and starting vector } x_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

```
Approximate x_infinity: [0. 0. 1.]
A x_infinity: [0. 0. 1.]

Eigenvalues: [1. 1. 1.]
```

Output corresponding to A_3 and x_3

Based on the resulting computations for the various transition matrices A_n and their corresponding limiting vectors x_∞, one can hypothesise that the limiting vector is always an eigenvector of the matrix A_n with eigenvalue 1. In each case, multiplying A_n by x_∞ gives the same vector back, showing this to be true. The other eigenvalues vary immensely, but they don't affect the final results, just how fast it converges. This supports the idea that for any *valid* transition matrix A_n, x_∞ will correspond to the eigenvalue 1.

(g) Google uses the following modified transition matrix, $M = (1-p)A + pB$, where $0 < p < 1$ is a parameter and $B = 1/3 \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$, explain what this modification achieves:

The modified matrix $M = (1-p)A + pB$ introduces a small chance of randomly jumping to any page, which prevents the system from getting stuck in dead ends or closed loops where some pages have no outgoing links or only link to each other. This modification ensures the system remains stable and always settles on a final, consistent ranking of all pages, regardless of the starting point.

```
use_google_matrix = input("\nDo you want to use the Google Matrix with
teleportation? (yes/no): ").strip().lower()
if use_google_matrix == 'yes':
    p = float(input("Enter the teleportation parameter p (0 <= p <= 1): "))
    if not (0 <= p <= 1):
        print("Error: p must be between 0 and 1.")
        return
    M = build_google_matrix(A, p)
    current_vector = x0.copy()
    history = [x0.copy()]
    for _ in range(N):
        next_vector = M @ current_vector
        assert is_probability_vector(next_vector)
        history.append(next_vector.copy())
        current_vector = next_vector
    x_infinity = current_vector
    print("\nUsing Google Matrix:")
    print("Approximate x_infinity:", x_infinity)
    print("M x_infinity:", M @ x_infinity)
```

Snippet of code execution of Google's modified transition matrix

```
Approximate x_infinity: [0.4286 0.2143 0.3571]
A x_infinity: [0.4286 0.2143 0.3571]

Eigenvalues: [ 1. +0.j     -0.5+0.2887j -0.5-0.2887j]

Eigenvectors:
 [[-0.7171+0.j     -0.7071+0.j     -0.7071-0.j    ]
 [-0.3586+0.j      0.5303+0.3062j  0.5303-0.3062j]
 [-0.5976+0.j      0.1768-0.3062j  0.1768+0.3062j]]

Do you want to use the Google Matrix with teleportation? (yes/no): yes
Enter the teleportation parameter p (0 <= p <= 1): .15

Using Google Matrix:
Approximate x_infinity: [0.4169 0.2272 0.3559]
M x_infinity: [0.4169 0.2272 0.3559]
```

Google matrix output corresponding to Figure 1

Whole Programme:

```
import numpy as np
import matplotlib.pyplot as plt
np.set_printoptions(precision=4, suppress=True)

# --- Helper Functions ---
def is_valid_transition_matrix(A):
    A = np.array(A)
    if (A < 0).any():  # allow zeros, reject negatives
        return False
    col_sums = np.sum(A, axis=0)
    return np.allclose(col_sums, np.ones(A.shape[1]))

def is_probability_vector(x):
    x = np.array(x)
    if (x < 0).any():
        return False
    return np.isclose(np.sum(x), 1)

def build_google_matrix(A, p):
    n = A.shape[0]
    B = np.ones((n, n)) / n
    M = (1 - p) * A + p * B
    return M

# --- Main Program ---
def main():
    # 1. User Input
    A = np.zeros((3, 3))
    print("Enter the elements of matrix A row by row (3x3 matrix):")
    for i in range(3):
        for j in range(3):
            while True:
                try:
                    user_input = input(f"Enter A[{i+1},{j+1}]: ")
                    A[i, j] = float(eval(user_input))
                    break
                except:
                    print("Invalid input. Enter a number or fraction like 1/2.")

    x0 = np.zeros(3)
    print("Enter the 3 elements of starting vector x0:")
    for i in range(3):
        while True:
            try:
                user_input = input(f"Enter x0[{i+1}]: ")
                x0[i] = float(eval(user_input))
                break
            except:
                print("Invalid input. Enter a number or fraction like 1/2.")

    # 2. Validate Matrix and Starting Vector
    if not is_valid_transition_matrix(A):
        print("Error: A is not a valid transition matrix.")
```

Whole Programme cont'd:

```
        return

    if not is_probability_vector(x0):
        print("Error: x0 is not a valid probability vector.")
        return

    # 3. Iterative Computation
    N = 1000  # Number of iterations
    history = [x0.copy()]
    current_vector = x0.copy()

    for _ in range(N):
        next_vector = A @ current_vector
        assert is_probability_vector(next_vector)
        history.append(next_vector.copy())
        current_vector = next_vector

    x_infinity = current_vector

    print("\nApproximate x_infinity:", x_infinity)
    print("A x_infinity:", A @ x_infinity)

    # 4. Plot convergence
    norms = [np.linalg.norm(x_infinity - x) for x in history]
    plt.plot(norms)
    plt.xlabel('Iteration')
    plt.ylabel('Norm ||x_infinity - x_n||')
    plt.title('Convergence to x_infinity')
    plt.grid(True)
    plt.show()

    # 5. Eigenvalues and Eigenvectors
    eigenvalues, eigenvectors = np.linalg.eig(A)
    print("\nEigenvalues:", np.round(eigenvalues, 4))
    print("\nEigenvectors:\n", np.round(eigenvectors, 4))

    # Using Google Matrix
    use_google_matrix = input("\nDo you want to use the Google Matrix with teleportation?
(yes/no): ").strip().lower()
    if use_google_matrix == 'yes':
        p = float(input("Enter the teleportation parameter p (0 <= p <= 1): "))
        if not (0 <= p <= 1):
            print("Error: p must be between 0 and 1.")
            return
        M = build_google_matrix(A, p)
        current_vector = x0.copy()
        history = [x0.copy()]
        for _ in range(N):
            next_vector = M @ current_vector
            assert is_probability_vector(next_vector)
            history.append(next_vector.copy())
            current_vector = next_vector
        x_infinity = current_vector
        print("\nUsing Google Matrix:")
        print("Approximate x_infinity:", x_infinity)
        print("M x_infinity:", M @ x_infinity)

if __name__ == "__main__":
    main()
```